

JUnit 5, three levels deep

Every annotation and assertion below is explained with the same running project, so each example builds on code you already know: **RentIt**, a small car-rental service. Read Level 1 in one sitting; keep Levels 2 and 3 as a reference.

LEVEL 1 · 1–2 HOURS

The essentials

Enough for most university assignments.

LEVEL 2 · 1–2 DAYS

Junior developer

Enough for most real-world projects.

LEVEL 3 · ONGOING

Professional testing

Tools that work alongside JUnit.

The running example – RentIt

Two classes are all you need to follow every example in this guide. `Car` is the data; `RentalAgency` is the class under test.

```
public class Car {
    private final String plate;      // "AB-123"
    private final String model;      // "Toyota Corolla"
    private final double dailyRate;  // price per day
    private boolean rented;
    // constructor, getters, isRented(), markRented(), markReturned()
}

public class RentalAgency {
    private final List<Car> fleet = new ArrayList<>();
    public void addCar(Car car) { ... }
    public Car findByPlate(String plate) { ... }      // null if not found
    public double rent(String plate, int days) { ... } // returns total price;
                                                    // throws if already rented
    public void returnCar(String plate) { ... }
    public int availableCount() { ... }
}
```

The essentials

Eleven building blocks. After this, you can write a complete test class for RentalAgency.

1.01 @Test

Marks a method as a test. JUnit finds every `@Test` method in the class and runs each one independently – no `main()` needed. The method must be non-private and return `void`.

```
class RentalAgencyTest {

    @Test
    void rentingChargesDailyRateTimesDays() {
        RentalAgency agency = new RentalAgency();
        agency.addCar(new Car("AB-123", "Toyota Corolla", 40.0));

        double price = agency.rent("AB-123", 3);

        assertEquals(120.0, price);
    }
}
```

WHEN TO USE – On every test method, always. One `@Test` = one behavior you want to verify.

1.02 assertEquals(expected, actual)

Fails the test unless the two values are equal. The *expected* value comes first – get the order right or failure messages read backwards. For doubles, add a third tolerance argument.

```
@Test
void weekRentalOfCorollaCosts280() {
    agency.addCar(new Car("AB-123", "Toyota Corolla", 40.0));

    assertEquals(280.0, agency.rent("AB-123", 7), 0.001); // tolerance for doubles
    assertEquals("Toyota Corolla", agency.findByPlate("AB-123").getModel());
}
```

WHEN TO USE – Whenever a method returns a value you can predict: prices, counts, names. Your most-used assertion by far.

1.03 assertTrue(condition)

Fails unless the condition is `true`. Use it for yes/no state, not for comparing values (that is `assertEquals`'s job – it gives better failure messages).

```
@Test
void rentedCarIsMarkedAsRented() {
    agency.addCar(new Car("AB-123", "Toyota Corolla", 40.0));

    agency.rent("AB-123", 3);

    assertTrue(agency.findByPlate("AB-123").isRented());
}
```

WHEN TO USE – Checking boolean state: is the car rented, is the fleet empty, does a plate exist.

1.04 `assertFalse(condition)`

The mirror of `assertTrue`: fails unless the condition is `false`. Prefer it over `assertTrue(!x)` – it reads better and fails better.

```
@Test
void returnedCarIsAvailableAgain() {
    agency.addCar(new Car("AB-123", "Toyota Corolla", 40.0));
    agency.rent("AB-123", 3);

    agency.returnCar("AB-123");

    assertFalse(agency.findByPlate("AB-123").isRented());
}
```

WHEN TO USE – Verifying something did *not* happen or a state was cleared – like a car becoming available after return.

1.05 `assertNull(value)`

Fails unless the value is `null`. Useful when your API uses `null` to mean "not found".

```
@Test
void unknownPlateReturnsNull() {
    assertNull(agency.findByPlate("ZZ-999")); // never added to the fleet
}
```

WHEN TO USE – Lookups that legitimately return nothing: a plate that isn't in the fleet.

1.06 `assertNotNull(value)`

Fails if the value is `null`. Often the first line of a test – guard against a `NullPointerException` before asserting details.

```
@Test
void addedCarCanBeFound() {
    agency.addCar(new Car("CD-456", "Renault Clio", 30.0));

    Car found = agency.findByPlate("CD-456");

    assertNotNull(found);
    assertEquals("Renault Clio", found.getModel());
}
```

WHEN TO USE – Before calling methods on a looked-up object, so a missing car fails clearly instead of crashing.

1.07 `@BeforeEach`

Runs before *every* test in the class. Use it to build a fresh, known starting state so tests never depend on each other. Notice how the examples above stopped repeating `new RentalAgency()` – this is why.

```

class RentalAgencyTest {
    private RentalAgency agency;

    @BeforeEach
    void setUp() { // runs again before EACH @Test
        agency = new RentalAgency();
        agency.addCar(new Car("AB-123", "Toyota Corolla", 40.0));
        agency.addCar(new Car("CD-456", "Renault Clio", 30.0));
    }
}

```

WHEN TO USE – As soon as two tests share setup. Every test gets its own agency – a car rented in one test is not rented in the next.

1.08 @AfterEach

Runs after every test, even one that failed. Use it to release resources – files, connections, temp data. For pure in-memory objects like our agency it's rarely needed.

```

@AfterEach
void tearDown() {
    agency.closeRentalLog(); // e.g. a file the agency writes contracts to
}

```

WHEN TO USE – Only when a test leaves something open or written behind. No resources, no @AfterEach.

1.09 @ParameterizedTest

Replaces @Test and runs the same method once per input. The inputs come from a source annotation – the two you need first are below. The method takes parameters; each run reports separately.

WHEN TO USE – When you catch yourself copy-pasting a test and changing only the numbers.

1.10 @ValueSource

The simplest source: one array of literals (ints, strings, doubles...), one parameter per run.

```

@ParameterizedTest@ValueSource(ints = {1, 7, 30}) // runs 3 times
void anyRentalLengthProducesPositivePrice(int days) {
    double price = agency.rent("AB-123", days);
    assertTrue(price > 0);
}

```

WHEN TO USE – Testing one rule against many values of a *single* input: weekend rental, week, month.

1.11 @CsvSource

Each string is one run, with comma-separated values mapped to the method's parameters – so you can pair inputs with their expected results.

```
@ParameterizedTest
@CsvSource({
    "1, 40.0",    // days, expected price (Corolla @ 40.0/day)
    "3, 120.0",
    "7, 280.0"
})
void priceIsDailyRateTimesDays(int days, double expected) {
    assertEquals(expected, agency.rent("AB-123", days), 0.001);
}
```

WHEN TO USE – Input → expected-output tables, like a pricing table. If the CSV grows past ~10 rows, look up `@CsvFileSource`.

Junior developer level

Structure, failure cases and control over when tests run. Enough for most real-world projects.

2.01 `assertThrows()`

Asserts that a piece of code throws a specific exception – and hands the exception back so you can check its message. Without it, a thrown exception just crashes the test.

```
@Test
void rentingAnAlreadyRentedCarThrows() {
    agency.rent("AB-123", 3);

    IllegalStateException ex = assertThrows(
        IllegalStateException.class,
        () -> agency.rent("AB-123", 2)    // the code that must fail
    );
    assertEquals("Car AB-123 is already rented", ex.getMessage());
}
```

WHEN TO USE – Every error path: double-renting, negative days, returning a car that was never rented. Error paths deserve tests as much as happy paths.

2.02 `assertAll()`

Runs a group of assertions and reports *all* failures, not just the first. Normally a test stops at the first failed assertion, hiding the rest.

```
@Test
void rentingUpdatesEverythingAtOnce() {
    double price = agency.rent("AB-123", 3);

    assertAll("state after renting the Corolla",
        () -> assertEquals(120.0, price, 0.001),
        () -> assertTrue(agency.findByPlate("AB-123").isRented()),
        () -> assertEquals(1, agency.availableCount())    // only the Clio left
    );
}
```

WHEN TO USE – When one action changes several things and you want the full damage report in one run.

2.03 `@Nested`

Groups related tests into inner classes, each with its own extra setup. The class structure becomes the documentation: "when the car is rented, then...".

```

class RentalAgencyTest {
    // @BeforeEach setUp() as before...

    @Nested
    class WhenCarIsRented {
        @BeforeEach
        void rentIt() { agency.rent("AB-123", 3); }    // runs AFTER outer setUp()

        @Test void itCannotBeRentedAgain() { ... }
        @Test void availableCountDrops() { ... }
        @Test void returningFreesIt() { ... }
    }
}

```

WHEN TO USE – When a test class grows past ~10 tests, or several tests share a scenario ("fleet is empty", "car is rented").

2.04 @DisplayName

Gives a test (or class) a human-readable name in reports – spaces, punctuation, anything – instead of the method name.

```

@DisplayName("A rented car cannot be rented a second time")
@Test
void doubleRentThrows() {
    agency.rent("AB-123", 3);
    assertThrows(IllegalStateException.class, () -> agency.rent("AB-123", 1));
}

```

WHEN TO USE – When the method name can't carry the whole story, and on @Nested classes for readable report trees.

2.05 @BeforeAll / @AfterAll

Run *once* per class – before the first test and after the last – not around each test. The methods must be **static**. Use for expensive shared setup that tests only read, never modify.

```

class PricingTest {
    private static PricingTable seasonalRates;

    @BeforeAll
    static void loadRates() {                // once, before all tests
        sc-camel-seasonal-rates = PricingTable.fromFile("rates-2026.csv");
    }

    @AfterAll
    static void report() {                   // once, after all tests
        System.out.println("Pricing tests done");
    }
}

```

WHEN TO USE – Loading a rate file, starting a shared server. Mutable state like our agency stays in @BeforeEach – shared mutable state makes tests depend on order.

2.06 Assumptions – @Disabled, @EnabledOnOs...

These *skip* tests instead of failing them. `@Disabled` turns a test off with a reason; `@EnabledOnOs` / `@DisabledOnOs` gate by platform; `assumeTrue()` aborts mid-test when a runtime condition isn't met. Skipped tests show as skipped, not green – you keep the honesty.

```
@Disabled("bug #42: price rounding – fix in progress")
@Test
void longRentalGetsDiscount() { ... }

@EnabledOnOs(OS.WINDOWS)           // contract PDFs use a Windows-only printer API
@Test
void contractPrintsOnWindows() { ... }

@Test
void syncsWithHeadOfficeFleet() {
    assumeTrue(headOffice.isReachable()); // skip (not fail) when offline
    agency.syncFleet(headOffice);
    assertEquals(headOffice.fleetSize(), agency.availableCount());
}
```

WHEN TO USE – Known bugs awaiting a fix, platform-specific behavior, tests needing an external service. Never delete a failing test – disable it with a reason.

2.07 @Tag

Labels tests so the build tool can include or exclude groups – e.g. run fast tests on every save, slow ones only in CI.

```
@Tag("slow")
@Test
void monthlyRevenueReportOverFullFleet() { ... } // iterates 10,000 rentals

// Gradle: run everything except the slow ones
test { useJUnitPlatform { excludeTags "slow" } }
```

WHEN TO USE – Once the suite is slow enough that you stop running it – split "fast" / "slow" / "integration".

Professional testing

Tools that work alongside JUnit. Skim now; learn each when the project demands it.

3.01 Mockito – mocking dependencies PRIORITY *****

Real code has dependencies you can't run in a test – a payment gateway, an email sender. Mockito builds fake versions you script and inspect, so the test stays about *your* logic.

```
@Test
void rentingChargesTheCustomersCard() {
    PaymentGateway gateway = mock(PaymentGateway.class); // fake, no real bank
    when(gateway.charge("card-1", 120.0)).thenReturn(true);

    RentalAgency agency = new RentalAgency(gateway);
    agency.addCar(new Car("AB-123", "Toyota Corolla", 40.0));

    agency.rentAndPay("AB-123", 3, "card-1");

    verify(gateway).charge("card-1", 120.0); // exactly once, right amount
}
```

WHEN TO USE – Whenever the class under test talks to something external or slow. Learn this first – it unlocks testing real codebases.

3.02 Integration testing

Unit tests check classes alone; integration tests check that they work *together* – agency plus a real repository plus a real database, no mocks.

```
@Tag("integration")
@Test
void rentedCarSurvivesARestart() {
    CarRepository repo = new JdbcCarRepository(testDataSource);
    RentalAgency agency = new RentalAgency(repo);
    agency.rent("AB-123", 3);

    RentalAgency restarted = new RentalAgency(repo); // fresh instance, same DB
    assertTrue(restarted.findByPlate("AB-123").isRented());
}
```

WHEN TO USE – For the seams mocks can't cover: SQL that must actually run, transactions, serialization. Keep them few and tagged – they're slow.

3.03 Spring Boot testing

If RentIt becomes a web service, Spring's test support runs slices of the app: `@WebMvcTest` boots only the web layer and `MockMvc` fires fake HTTP requests at it.

```

@WebMvcTest(RentalController.class)
class RentalControllerTest {
    @Autowired MockMvc mvc;
    @MockBean RentalAgency agency;    // Mockito mock, injected by Spring

    @Test
    void listsAvailableCars() throws Exception {
        when(agency.availableCount()).thenReturn(2);
        mvc.perform(get("/api/cars/available"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.count").value(2));
    }
}

```

WHEN TO USE – Only once you learn Spring. Note it composes what you already know: JUnit runs it, Mockito fakes the agency.

3.04 Testcontainers

Spins up a real database in Docker for the duration of a test class, then throws it away. Integration tests get production-grade PostgreSQL instead of an in-memory imitation.

```

@Testcontainers
class CarRepositoryIT {
    @Container
    static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16");

    @Test
    void savesAndReloadsTheFleet() {
        CarRepository repo = new JdbcCarRepository(db.getJdbcUrl());
        repo.save(new Car("AB-123", "Toyota Corolla", 40.0));
        assertNotNull(repo.findByPlate("AB-123"));
    }
}

```

WHEN TO USE – When integration tests need a real database and "works on my machine" isn't good enough. Requires Docker.

3.05 Code coverage with JaCoCo

Measures which lines your tests actually execute and reports the percentage per class. It tells you where tests are *missing* – not whether the existing ones are good.

```

// build.gradle
plugins { id 'jacoco' }
test { finalizedBy jacocoTestReport }

// then: ./gradlew test → build/reports/jacoco/.../index.html
// e.g. RentalAgency 92% – the uncovered 8% is returnCar()'s error path

```

WHEN TO USE – Once a suite exists, as a map of blind spots. Chase uncovered *branches*, not a vanity percentage.

The path

Write a full `RentalAgencyTest` with Level 1 alone → add error cases and `@Nested` structure with Level 2 → reach for Level 3 tools only when a real dependency forces you to. Mockito first; the rest when the project demands them.